

CCS6224

Network Security

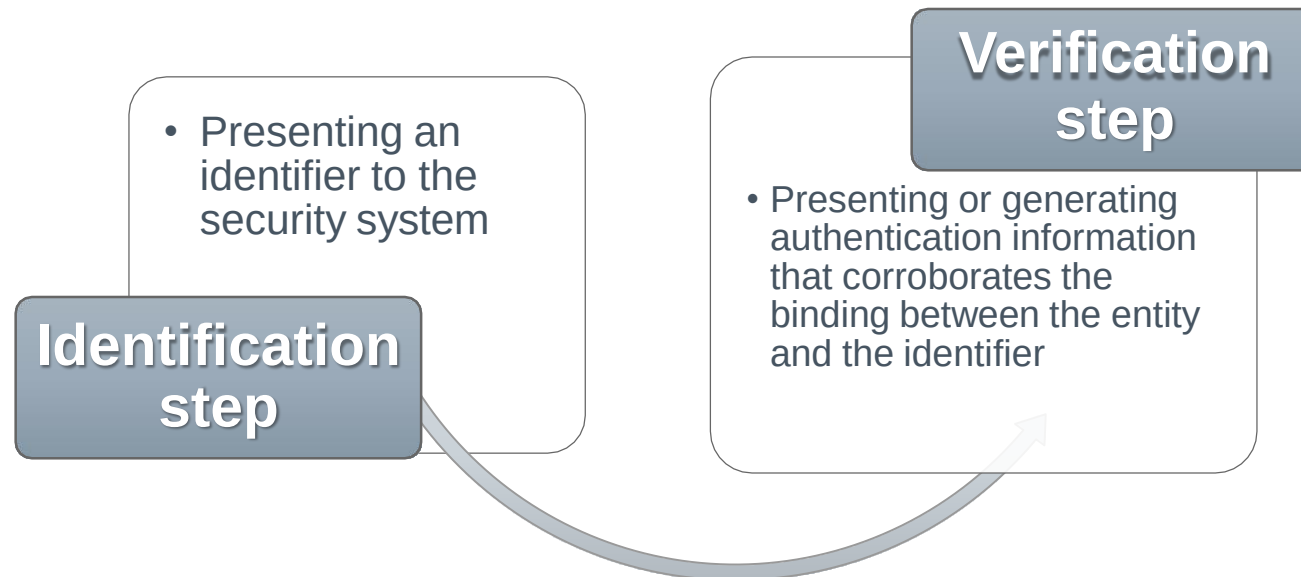
Lecture 7
Network Access Control (NAC)
Authentication Mechanism

Outline

- › Remote user-authentication principles
 - Mutual authentication
 - One-way authentication
- › Remote user-authentication using symmetric encryption
 - Mutual authentication
 - One-way authentication
- › Kerberos
 - Motivation
 - Kerberos V4 and V5
- › Federated identity management
 - Identity management
 - Identity federation
- › Personal identity verification
 - PIV system model
 - PIV documentation
 - PIV credentials and keys
 - authentication

Remote User-Authentication Principles

- › The process of verifying an identity claimed by or for a system entity
- › An authentication process consists of two steps:



Means of User Authentication

Something the individual knows

- Examples include a password, a personal identification number (PIN), or answers to a prearranged set of questions

Something the individual possesses

- Examples include cryptographic keys, electronic keycards, smart cards, and physical keys
- This is referred to as a token

There are four general means of authenticating a user's identity, which can be used alone or in combination

Something the individual is (static biometrics)

- Examples include recognition by fingerprint, retina, and face

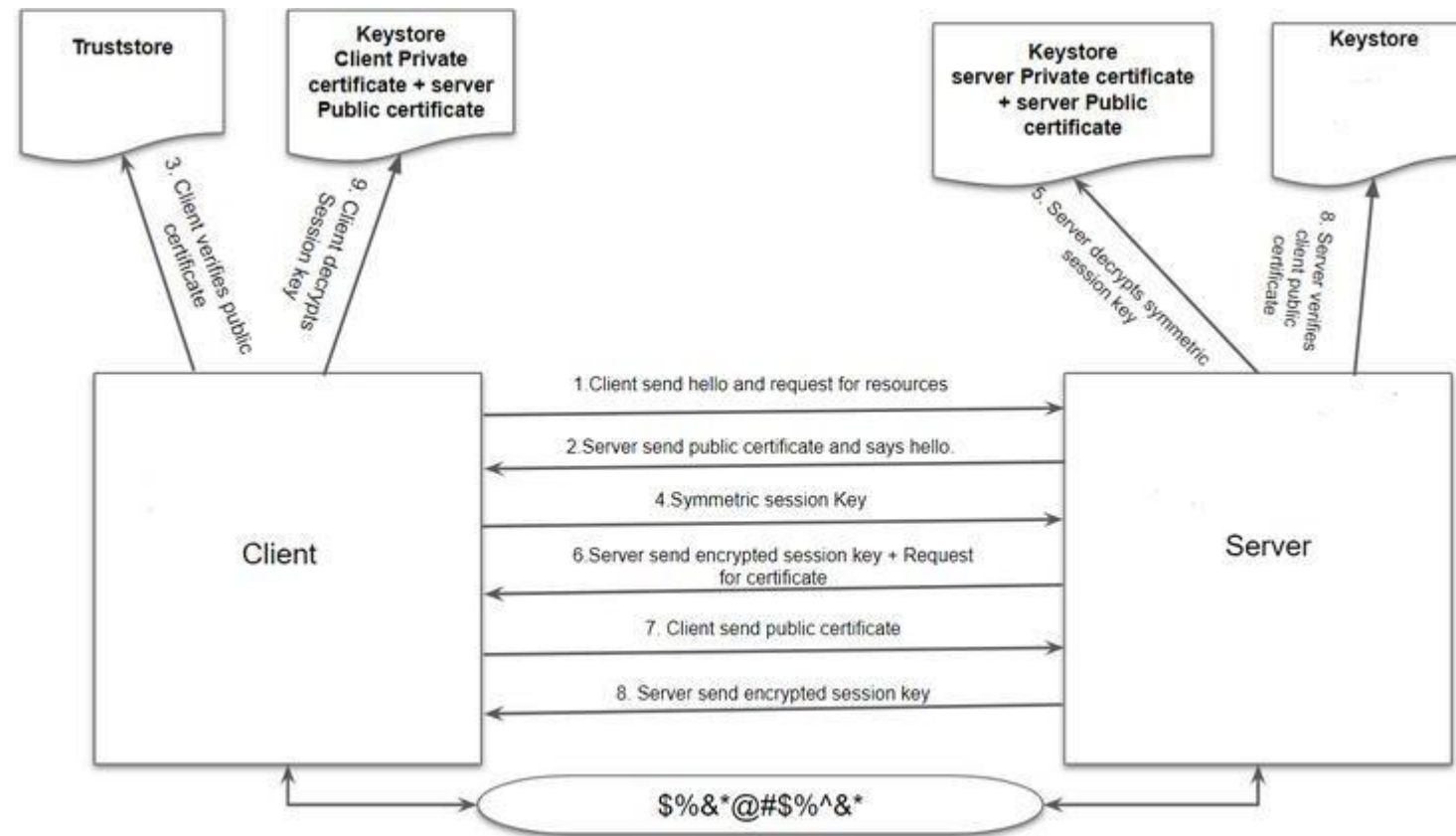
Something the individual does (dynamic biometrics)

- Examples include recognition by voice pattern, handwriting characteristics, and typing rhythm

- › For network-based user authentication, the most important methods involve cryptographic keys and something the individual knows, such as a password

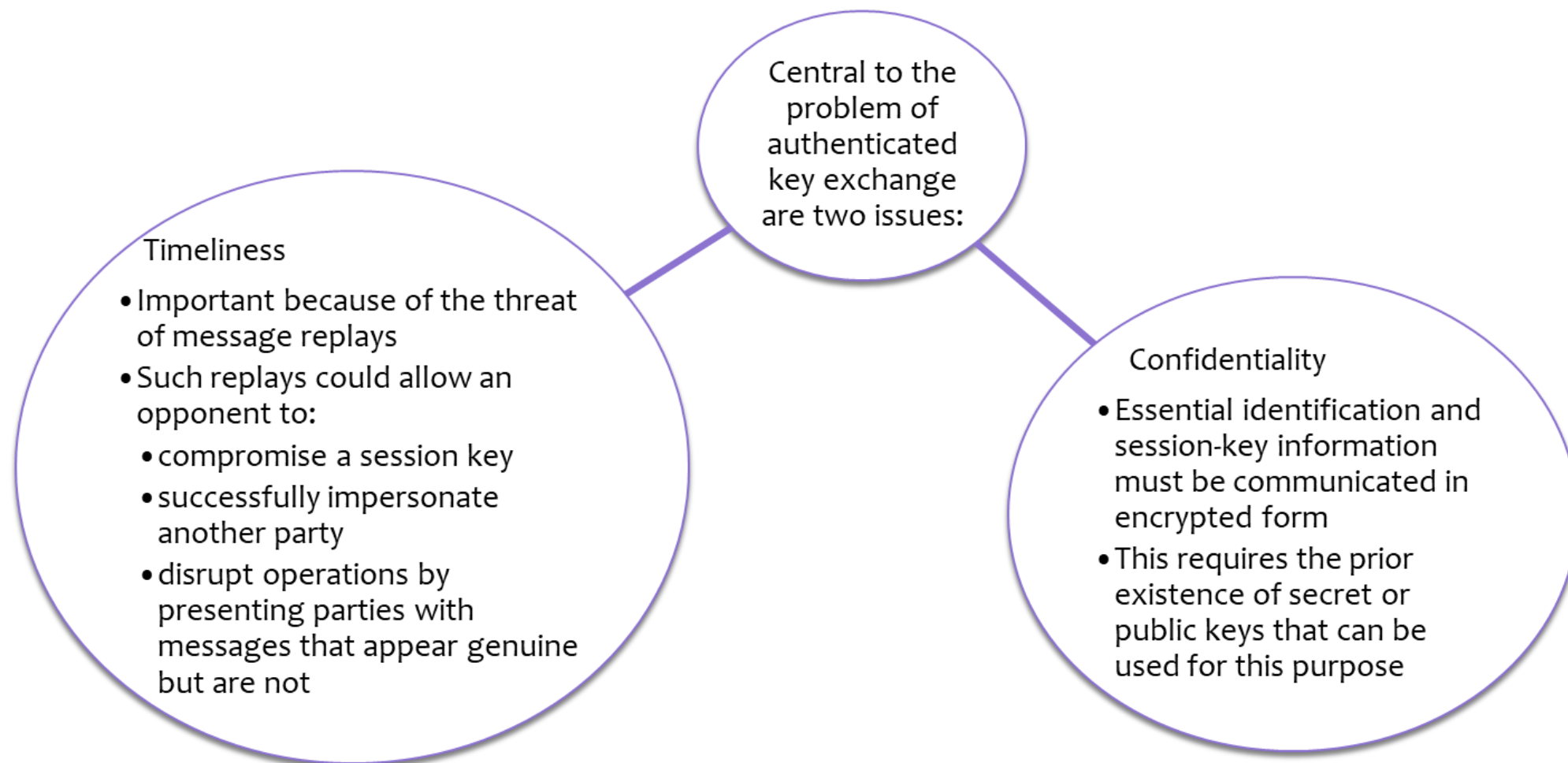
Mutual Authentication

- › Protocols which enable communicating parties to satisfy themselves mutually about each other's identity and to exchange session keys





Mutual Authentication



Replay Attacks

1. The simplest replay attack is one in which the opponent simply copies a message and replays it later
2. An opponent can replay a timestamped message within the valid time window
3. An opponent can replay a timestamped message within the valid time window, but in addition, the opponent suppresses the original message; thus, the repetition cannot be detected
4. Another attack involves a backward replay without modification and is possible if symmetric encryption is used and the sender cannot easily recognize the difference between messages sent and messages received on the basis of content

Approaches to Coping With Replay Attacks

- › Attach a sequence number to each message used in an authentication exchange
 - A new message is accepted only if its sequence number is in the proper order
 - Difficulty with this approach is that it requires each party to keep track of the last sequence number for each claimant it has dealt with
 - Generally not used for authentication and key exchange because of overhead
- Timestamps
 - Requires that clocks among the various participants be synchronized
 - Party A accepts a message as fresh only if the message contains a timestamp that, in A's judgment, is close enough to A's knowledge of current time
- Challenge/response
 - Party A, expecting a fresh message from B, first sends B a nonce (challenge) and requires that the subsequent message (response) received from B contain the correct nonce value



One-Way Authentication

One application for which encryption is growing in popularity is electronic mail (e-mail)

- Header of the e-mail message must be in the clear so that the message can be handled by the store-and-forward e-mail protocol, such as SMTP or X.400
- The e-mail message should be encrypted such that the mail-handling system is not in possession of the decryption key

A second requirement is that of authentication

- The recipient wants some assurance that the message is from the alleged sender

Remote User-Authentication Using Symmetric Encryption

A two-level hierarchy of symmetric keys can be used to provide confidentiality for communication in a distributed environment

- Strategy involves the use of a trusted key distribution center (KDC)
- Each party shares a secret key, known as a master key, with the KDC
- KDC is responsible for generating keys to be used for a short time over a connection between two parties and for distributing those keys using the master keys to protect the distribution

Suppress-Replay Attacks

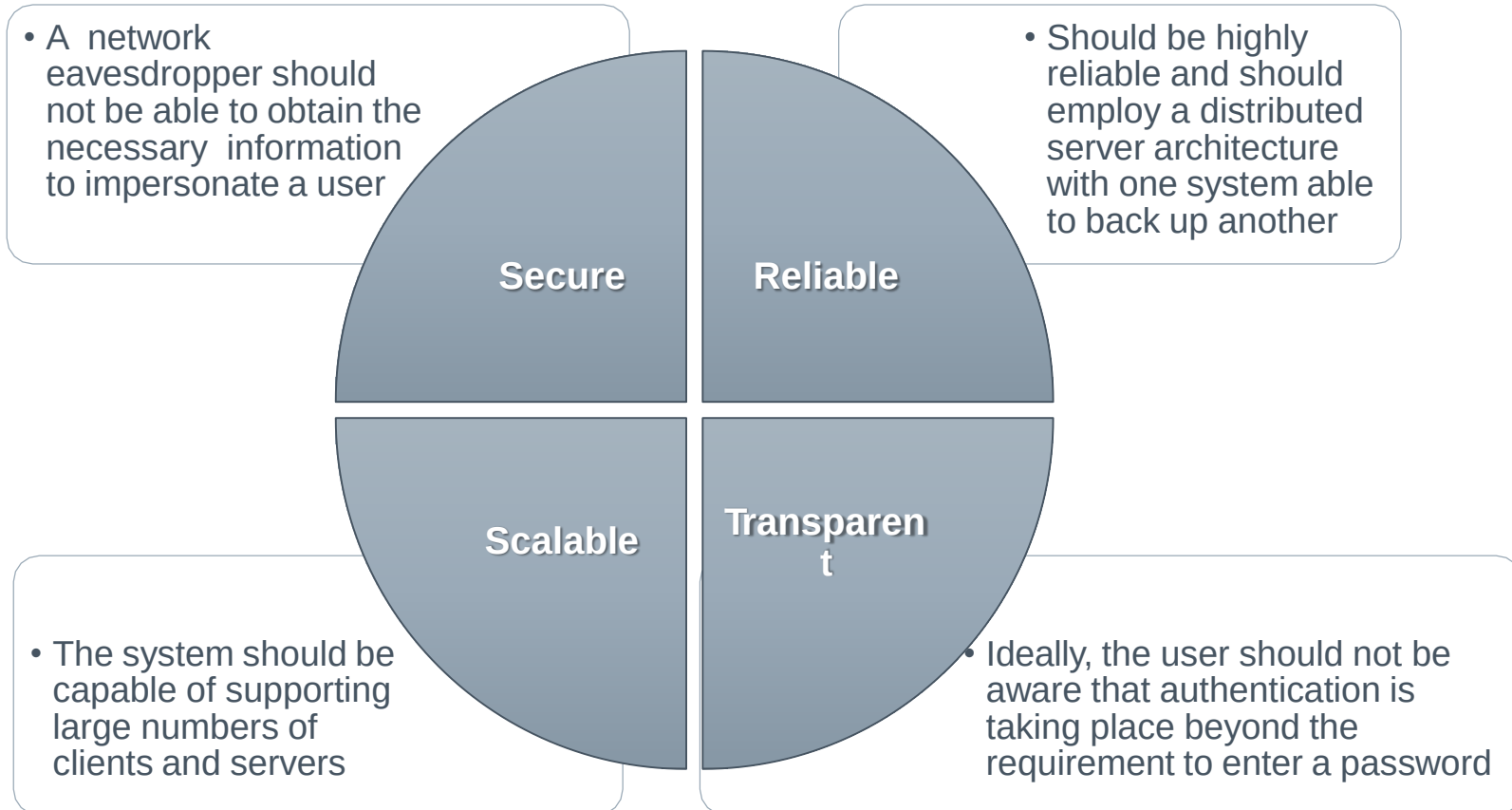
- › The Denning protocol requires reliance on clocks that are synchronized throughout the network
- › A risk involved is based on the fact that the distributed clocks can become unsynchronized as a result of sabotage on or faults in the clocks or the synchronization mechanism
- › The problem occurs when a sender's clock is ahead of the intended recipient's clock
 - An opponent can intercept a message from the sender and replay it later when the timestamp in the message becomes current at the recipient's site
 - Such attacks are referred to as *suppress-replay attacks*

Kerberos

- › Authentication service developed as part of Project Athena at MIT
- › A workstation cannot be trusted to identify its users correctly to network services
 - A user may gain access to a particular workstation and pretend to be another user operating from that workstation
 - A user may alter the network address of a workstation so that the requests sent from the altered workstation appear to come from the impersonated workstation
 - A user may eavesdrop on exchanges and use a replay attack to gain entrance to a server or to disrupt operations
- Kerberos provides a centralized authentication server whose function is to authenticate users to servers and servers to users
 - Relies exclusively on symmetric encryption, making no use of public-key encryption

Kerberos Requirements

- › The first published report on Kerberos listed the following requirements:





Kerberos Version 4

- › Makes use of DES to provide the authentication service
- › Authentication server (AS)
 - Knows the passwords of all users and stores these in a centralized database
 - Shares a unique secret key with each server
- › Ticket
 - Created once the AS accepts the user as authentic; contains the user's ID and network address and the server's ID
 - Encrypted using the secret key shared by the AS and the server
- Ticket-granting server (TGS)
 - Issues tickets to users who have been authenticated to AS
 - Each time the user requires access to a new service the client applies to the TGS using the ticket to authenticate itself
 - The TGS then grants a ticket for the particular service
 - The client saves each service-granting ticket and uses it to authenticate its user to a server each time a particular service is requested



The Version 4 Authentication Dialogue

The lifetime associated with the ticket-granting ticket creates a problem:

- If the lifetime is very short (e.g., minutes), the user will be repeatedly asked for a password
- If the lifetime is long (e.g., hours), then an opponent has a greater opportunity for replay

A network service (the TGS or an application service) must be able to prove that the person using a ticket is the same person to whom that ticket was issued

Servers need to authenticate themselves to users

Summary of Kerberos Version 4 Message Exchanges

(1) $C \rightarrow AS$ $ID_c \parallel ID_{tgs} \parallel TS_1$
(2) $AS \rightarrow C$ $E(K_c, [K_{c,tgs} \parallel ID_{tgs} \parallel TS_2 \parallel Lifetime_2 \parallel Ticket_{tgs}])$
 $Ticket_{tgs} = E(K_{tgs}, [K_{c,tgs} \parallel ID_c \parallel AD_c \parallel ID_{tgs} \parallel TS_2 \parallel Lifetime_2])$

(a) Authentication Service Exchange to obtain ticket-granting ticket

(3) $C \rightarrow TGS$ $ID_v \parallel Ticket_{tgs} \parallel Authenticator_c$
(4) $TGS \rightarrow C$ $E(K_{c,tgs}, [K_{c,v} \parallel ID_v \parallel TS_4 \parallel Ticket_v])$
 $Ticket_{tgs} = E(K_{tgs}, [K_{c,tgs} \parallel ID_c \parallel AD_c \parallel ID_{tgs} \parallel TS_2 \parallel Lifetime_2])$
 $Ticket_v = E(K_v, [K_{c,v} \parallel ID_c \parallel AD_c \parallel ID_v \parallel TS_4 \parallel Lifetime_4])$
 $Authenticator_c = E(K_{c,tgs}, [ID_c \parallel AD_c \parallel TS_3])$

(b) Ticket-Granting Service Exchange to obtain service-granting ticket

(5) $C \rightarrow V$ $Ticket_v \parallel Authenticator_c$
(6) $V \rightarrow C$ $E(K_{c,v}, [TS_5 + 1])$ (for mutual authentication)
 $Ticket_v = E(K_v, [K_{c,v} \parallel ID_c \parallel AD_c \parallel ID_v \parallel TS_4 \parallel Lifetime_4])$
 $Authenticator_c = E(K_{c,v}, [ID_c \parallel AD_c \parallel TS_5])$

(c) Client/Server Authentication Exchange to obtain service

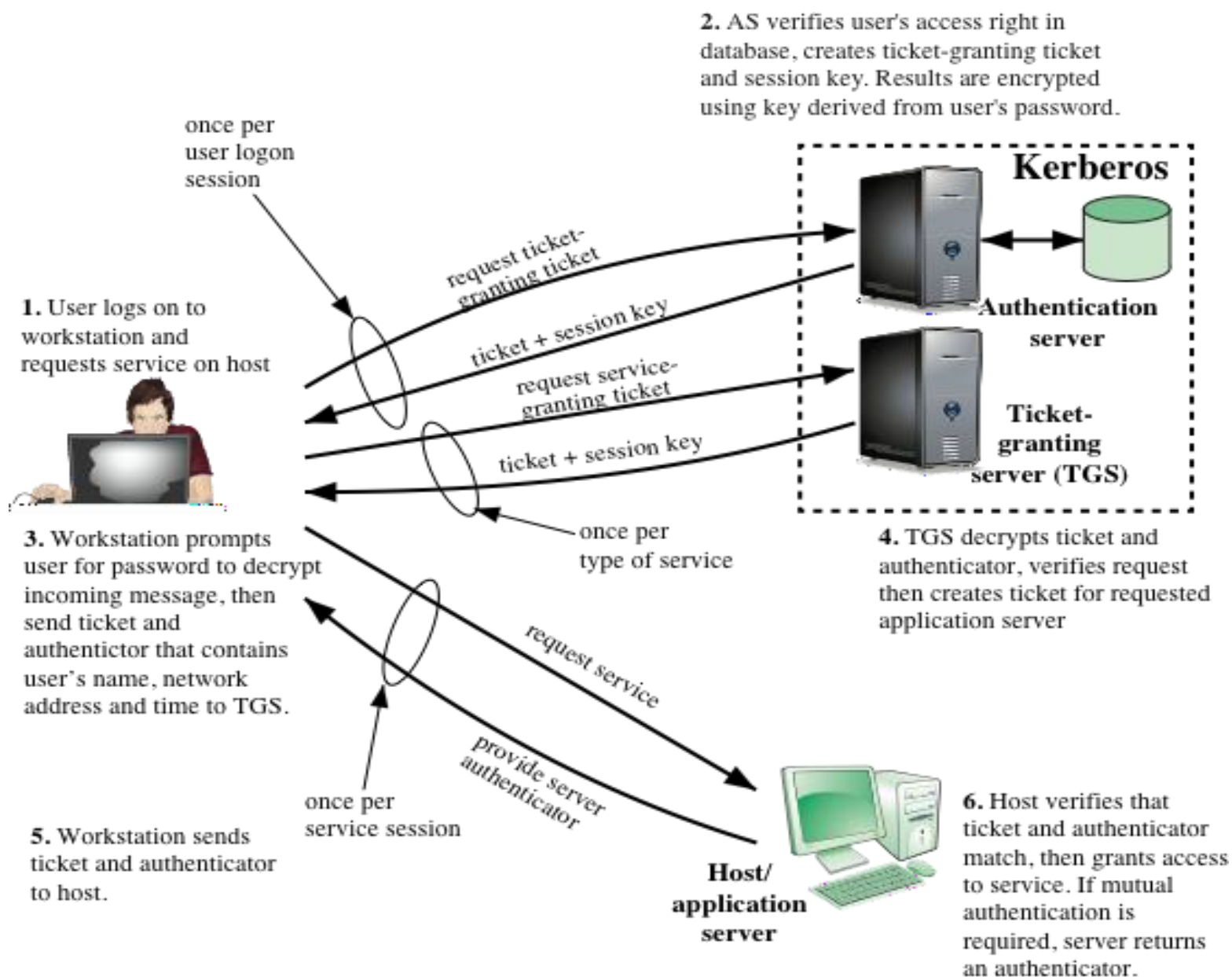


Figure 15.1 Overview of Kerberos

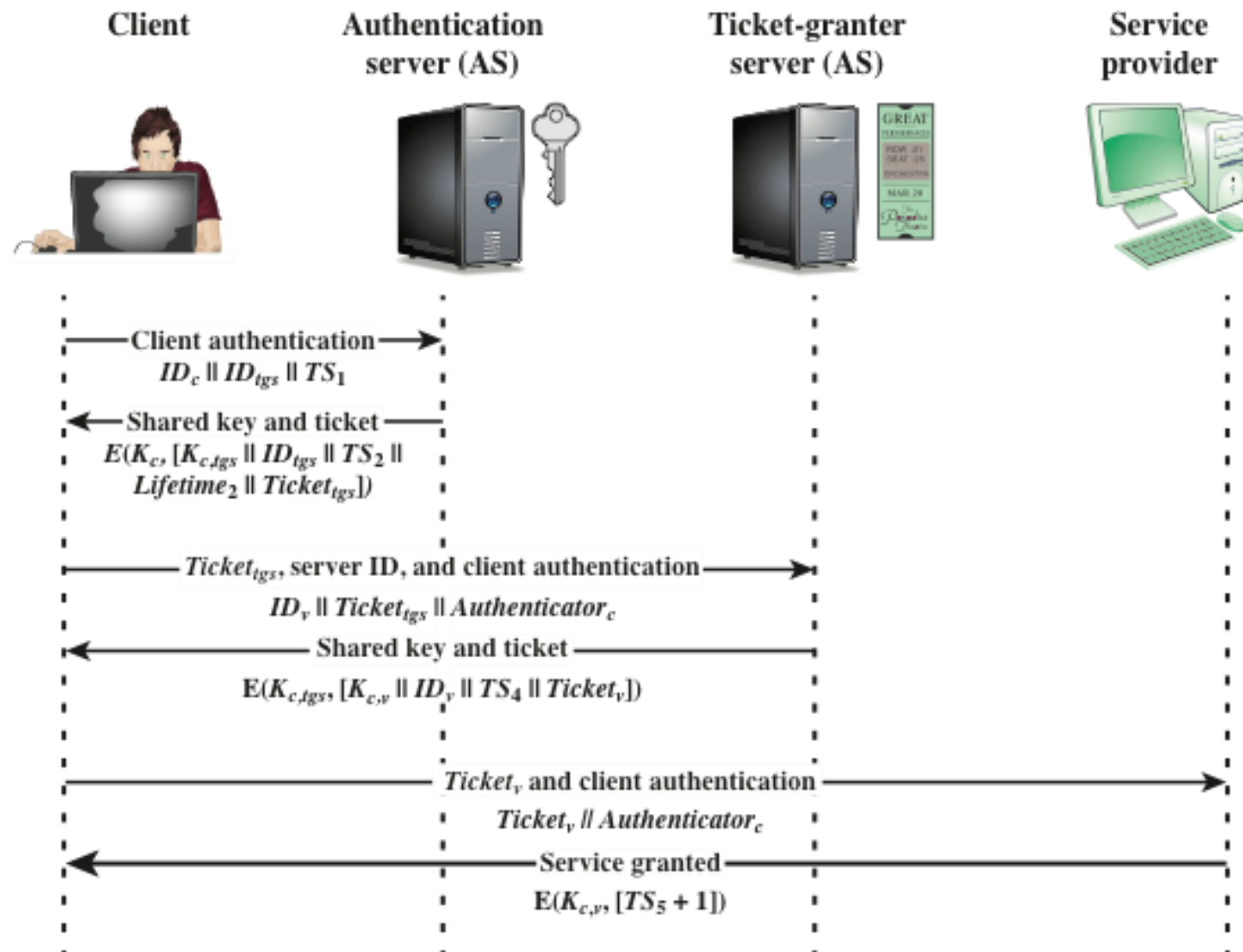


Figure 15.2 Kerberos Exchanges

Table 15.2 Rationale for the Elements of the Kerberos Version 4 Protocol
(page 1 of 3)

Message (1)	Client requests ticket-granting ticket.
ID_C	Tells AS identity of user from this client.
ID_{tgs}	Tells AS that user requests access to TGS.
TS_1	Allows AS to verify that client's clock is synchronized with that of AS.
Message (2)	AS returns ticket-granting ticket.
K_c	Encryption is based on user's password, enabling AS and client to verify password, and protecting contents of message (2).
$K_{c,tgs}$	Copy of session key accessible to client created by AS to permit secure exchange between client and TGS without requiring them to share a permanent key.
ID_{tgs}	Confirms that this ticket is for the TGS.
TS_2	Informs client of time this ticket was issued.
$Lifetime_2$	Informs client of the lifetime of this ticket.
$Ticket_{tgs}$	Ticket to be used by client to access TGS.

Message (3)	Client requests service-granting ticket.
ID_V	Tells TGS that user requests access to server V.
$Ticket_{tgs}$	Assures TGS that this user has been authenticated by AS.
$Authenticator_c$	Generated by client to validate ticket .
Message (4)	TGS returns service-granting ticket.
$K_{c,tgs}$	Key shared only by C and TGS protects contents of message (4).
$K_{c,v}$	Copy of session key accessible to client created by TGS to permit secure exchange between client and server without requiring them to share a permanent key.
ID_V	Confirms that this ticket is for server V.
TS_4	Informs client of time this ticket was issued.
$Ticket_V$	Ticket to be used by client to access server V.
$Ticket_{tgs}$	Reusable so that user does not have to reenter password.
K_{tgs}	Ticket is encrypted with key known only to AS and TGS, to prevent Tampering.
$K_{c,tgs}$	Copy of session key accessible to TGS used to decrypt authenticator, thereby authenticating ticket.
ID_C	Indicates the rightful owner of this ticket.
AD_C	Prevents use of ticket from workstation other than one that initially requested the ticket.
ID_{tgs}	Assures server that it has decrypted ticket properly.
TS_2	Informs TGS of time this ticket was issued.
$Lifetime_2$	Prevents replay after ticket has expired.
$Authenticator_c$	Assures TGS that the ticket presenter is the same as the client for whom the ticket was issued has very short lifetime to prevent replay.
$K_{c,tgs}$	Authenticator is encrypted with key known only to client and TGS, to prevent tampering.
ID_C	Must match ID in ticket to authenticate ticket.
AD_C	Must match address in ticket to authenticate ticket.
TS_3	Informs TGS of time this authenticator was generated.

Message (5)	Client requests service.
$Ticket_V$	Assures server that this user has been authenticated by AS.
$Authenticator_c$	Generated by client to validate ticket.
Message (6)	Optional authentication of server to client.
$K_{c,v}$	Assures C that this message is from V.
$TS_5 + 1$	Assures C that this is not a replay of an old reply.
$Ticket_V$	Reusable so that client does not need to request a new ticket from TGS for each access to the same server.
K_V	Ticket is encrypted with key known only to TGS and server, to prevent Tampering.
$K_{c,v}$	Copy of session key accessible to client; used to decrypt authenticator, thereby authenticating ticket.
ID_C	Indicates the rightful owner of this ticket.
AD_C	Prevents use of ticket from workstation other than one that initially requested the ticket.
ID_V	Assures server that it has decrypted ticket properly.
TS_4	Informs server of time this ticket was issued.
$Lifetime_4$	Prevents replay after ticket has expired.
$Authenticator_c$	Assures server that the ticket presenter is the same as the client for whom the ticket was issued; has very short lifetime to prevent replay.
$K_{c,v}$	Authenticator is encrypted with key known only to client and server, to prevent tampering.
ID_C	Must match ID in ticket to authenticate ticket.
AD_c	Must match address in ticket to authenticate ticket.
TS_5	Informs server of time this authenticator was generated.

Kerberos Realms and Multiple Kerber

- › A full-service Kerberos environment consisting of a Kerberos server, a number of clients, and a number of application servers requires that:
 - The Kerberos server must have the user ID and hashed passwords of all participating users in its database; all users are registered with the Kerberos server
 - The Kerberos server must share a secret key with each server; all servers are registered with the Kerberos server
 - The Kerberos server in each interoperating realm shares a secret key with the server in the other realm; the two Kerberos servers are registered with each other

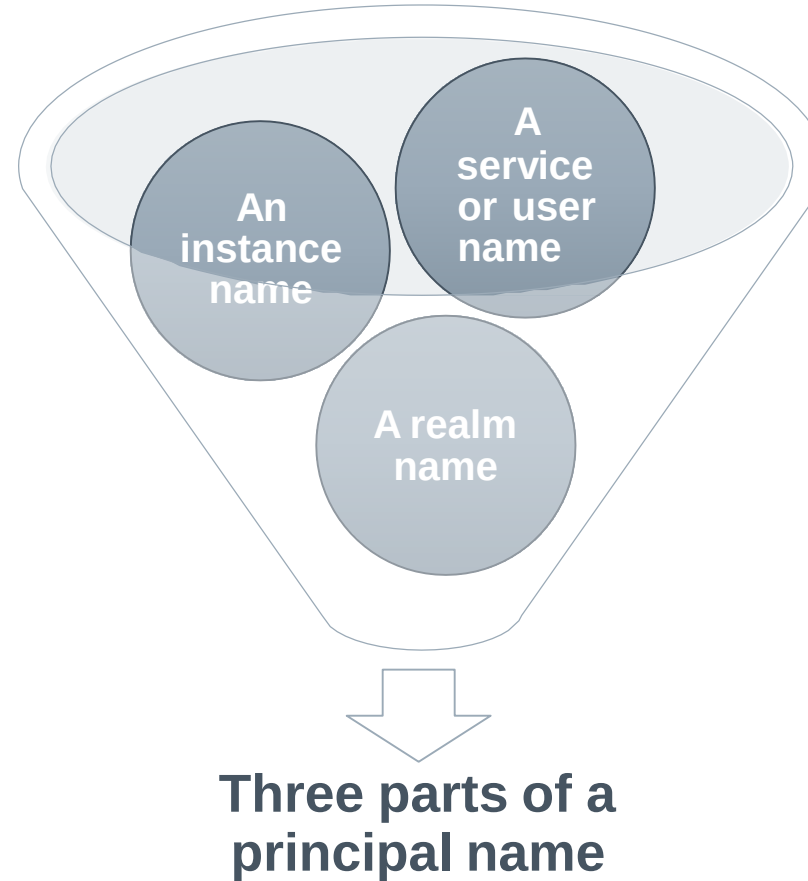


Kerberos Realm

- › A set of managed nodes that share the same Kerberos database
- › The database resides on the Kerberos master computer system, which should be kept in a physically secure room
- › A read-only copy of the Kerberos database might also reside on other Kerberos computer systems
- › All changes to the database must be made on the master computer system
- › Changing or accessing the contents of a Kerberos database requires the Kerberos master password

Kerberos Principal

- › A service or user that is known to the Kerberos system
- › Identified by its principal name



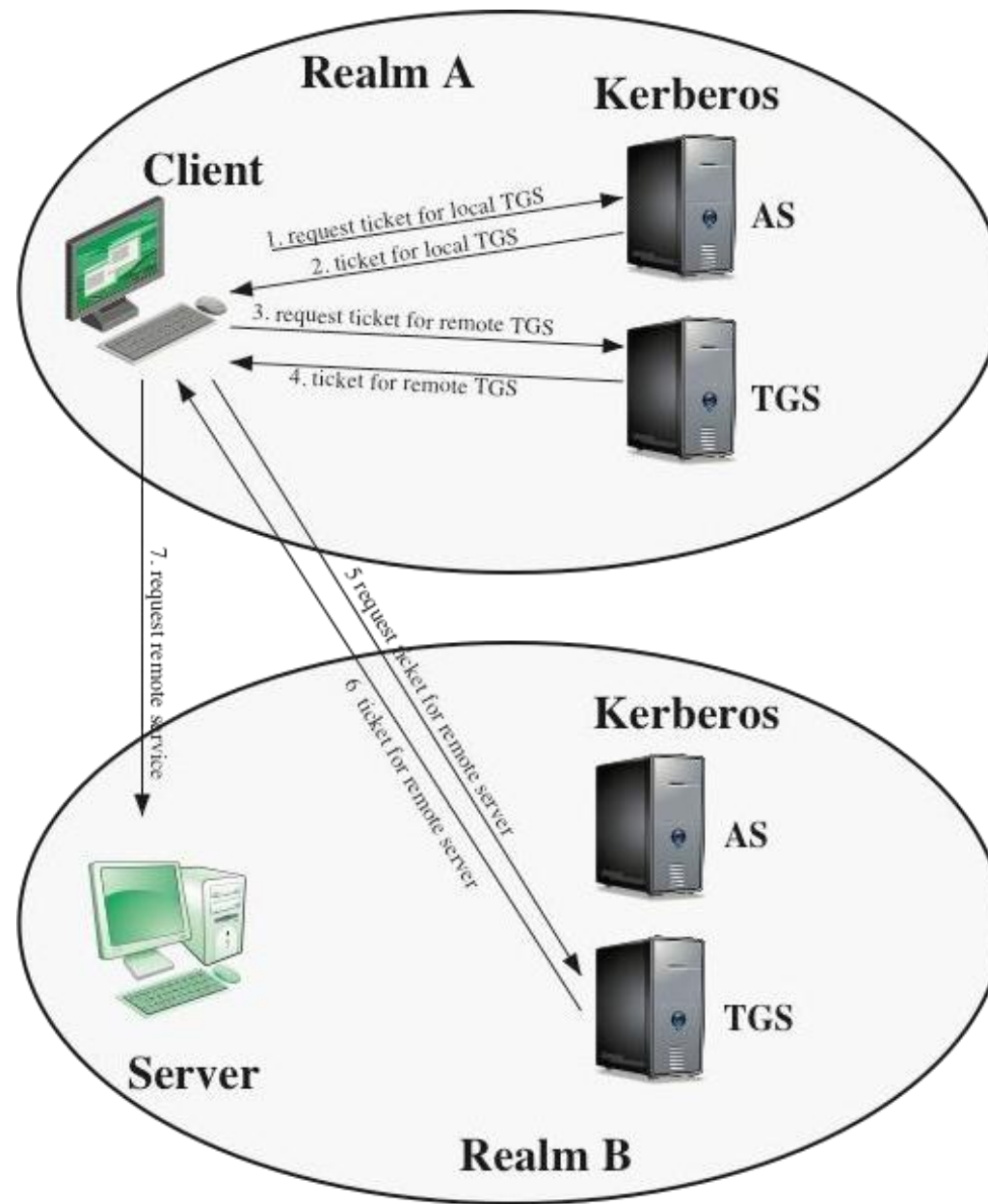


Figure 15.3 Request for Service in Another Realm

Differences Between Versions 4 and 5

Version 5 is intended to address the limitations of version 4 in two areas:

Environmental shortcomings

- Encryption system dependence
- Internet protocol dependence
- Message byte ordering
- Ticket lifetime
- Authentication forwarding
- Interrealm authentication

Technical deficiencies

- Double encryption
- PCBC encryption
- Session keys
- Password attacks

Summary of Kerberos Version 5 Message Exchanges

(1) $C \rightarrow AS$ $Options \parallel ID_C \parallel Realm_C \parallel ID_{TGS} \parallel Times \parallel Nonce_1$
(2) $AS \rightarrow C$ $Realm_C \parallel IDC \parallel Ticket_{TGS} \parallel E(K_c, [K_{c,TGS} \parallel Times \parallel Nonce_1 \parallel Realm_{TGS} \parallel ID_{TGS}])$
 $Ticket_{TGS} = E(K_{TGS}, [Flags \parallel K_{c,TGS} \parallel Realm_C \parallel IDC \parallel ADC \parallel Times])$

(a) Authentication Service Exchange to obtain ticket-granting ticket

(3) $C \rightarrow TGS$ $Options \parallel ID_V \parallel Times \parallel Nonce_2 \parallel Ticket_{TGS} \parallel Authenticator_C$
(4) $TGS \rightarrow C$ $Realm_C \parallel IDC \parallel Ticket_V \parallel E(K_{c,TGS}, [K_{c,V} \parallel Times \parallel Nonce_2 \parallel Realm_V \parallel ID_V])$
 $Ticket_{TGS} = E(K_{TGS}, [Flags \parallel K_{c,TGS} \parallel Realm_C \parallel IDC \parallel ADC \parallel Times])$
 $Ticket_V = E(K_V, [Flags \parallel K_{c,V} \parallel Realm_C \parallel IDC \parallel ADC \parallel Times])$
 $Authenticator_C = E(K_{c,TGS}, [ID_C \parallel Realm_C \parallel TS_1])$

(b) Ticket-Granting Service Exchange to obtain service-granting ticket

(5) $C \rightarrow V$ $Options \parallel Ticket_V \parallel Authenticator_C$
(6) $V \rightarrow C$ $E_{K_{C,V}} [TS_2 \parallel Subkey \parallel Seq\#]$
 $Ticket_V = E(K_V, [Flags \parallel K_{c,V} \parallel Realm_C \parallel IDC \parallel ADC \parallel Times])$
 $Authenticator_C = E(K_{c,V}, [ID_C \parallel Realm_C \parallel TS_2 \parallel Subkey \parallel Seq\#])$

(c) Client/Server Authentication Exchange to obtain service

Kerberos Version 5 Flags

INITIAL	This ticket was issued using the AS protocol and not issued based on a ticket-granting ticket.
PRE-AUTHENT	During initial authentication, the client was authenticated by the KDC before a ticket was issued.
HW-AUTHENT	The protocol employed for initial authentication required the use of hardware expected to be possessed solely by the named client.
RENEWABLE	Tells TGS that this ticket can be used to obtain a replacement ticket that expires at a later date.
MAY-POSTDATE	Tells TGS that a postdated ticket may be issued based on this ticket-granting ticket.
POSTDATED	Indicates that this ticket has been postdated; the end server can check the authtime field to see when the original authentication occurred.
INVALID	This ticket is invalid and must be validated by the KDC before use.
PROXIABLE	Tells TGS that a new service-granting ticket with a different network address may be issued based on the presented ticket.
PROXY	Indicates that this ticket is a proxy.
FORWARDABLE	Tells TGS that a new ticket-granting ticket with a different network address may be issued based on this ticket-granting ticket.
FORWARDED	Indicates that this ticket has either been forwarded or was issued based on authentication involving a forwarded ticket-granting ticket.

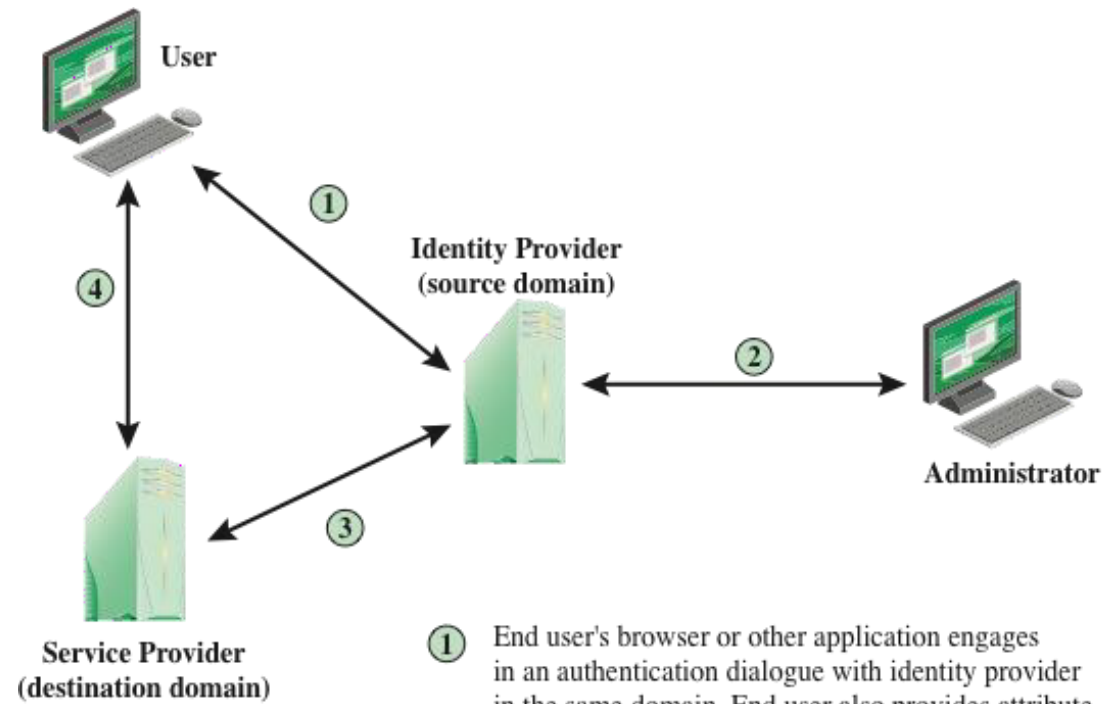


Federated Identity Management

- › Federated identity allows authorized users to access multiple applications and domains using a single set of credentials. It links a user's identity across multiple identity management systems so they can access different applications securely and efficiently.
- › When organizations implement federated identity solutions, their users can access web applications, partner websites, Active Directory, and other applications without logging in separately every time.

How Does Federated Identity Work?

- works on the basis of mutual trust relationships between a Service Provider (SP) such as an application vendor and an external party or Identity Provider (IdP).
- The IdP creates and manages user credentials and the SP and IdP agree on an authentication process. Multiple SPs can participate in a federated identity agreement with a single IdP. The IdP has mutual trust agreements with all these organizations.
- One example of federated identity is when a user logs into a third-party website by using their Gmail login credentials. With FIM, they don't have to create new credentials to access multiple websites that have a federated agreement with Google, such as:
YouTube, Fitbit, Waze, Picasa



- ① End user's browser or other application engages in an authentication dialogue with identity provider in the same domain. End user also provides attribute values associated with user's identity.
- ② Some attributes associated with an identity, such as allowable roles, may be provided by an administrator in the same domain.
- ③ A service provider in a remote domain, which the user wishes to access, obtains identity information, authentication information, and associated attributes from the identity provider in the source domain.
- ④ Service provider opens session with remote user and enforces access control restrictions based on user's identity and attributes.

Figure 15.5 Federated Identity Operation

π Key Standards

The Extensible Markup Language (XML)

A markup language that uses sets of embedded tags or labels to characterize text elements within a document so as to indicate their appearance, function, meaning, or context

The Simple Object Access Protocol (SOAP)

Enables applications to request services from one another with XML-based requests and receive responses as data formatted with XML

WS-Security

A set of SOAP extensions for implementing message integrity and confidentiality in Web services

Security Assertion Markup Language (SAML)

An XML-based language for the exchange of security information between online business partners

Benefits of Federated Identity

π

- Enhanced security
- Enhanced user experience
- Single-point provisioning
- Secure resource-sharing
- Cost savings

Personal Identity Verification

π

- › User authentication based on the possession of a smart card is becoming more widespread
 - Has the appearance of a credit card
 - Has an electronic interface
 - May use a variety of authentication protocols
- › A smart card contains within it an entire microprocessor, including processor, memory, and I/O ports
- › A smart card includes three types of memory:
 - Read-only memory (ROM) stores data that does not change during the card's life
 - Electronically erasable programmable ROM (EEPROM) holds application data and programs; also holds data that may vary with time
 - Random access memory (RAM) holds temporary data generated when applications are executed

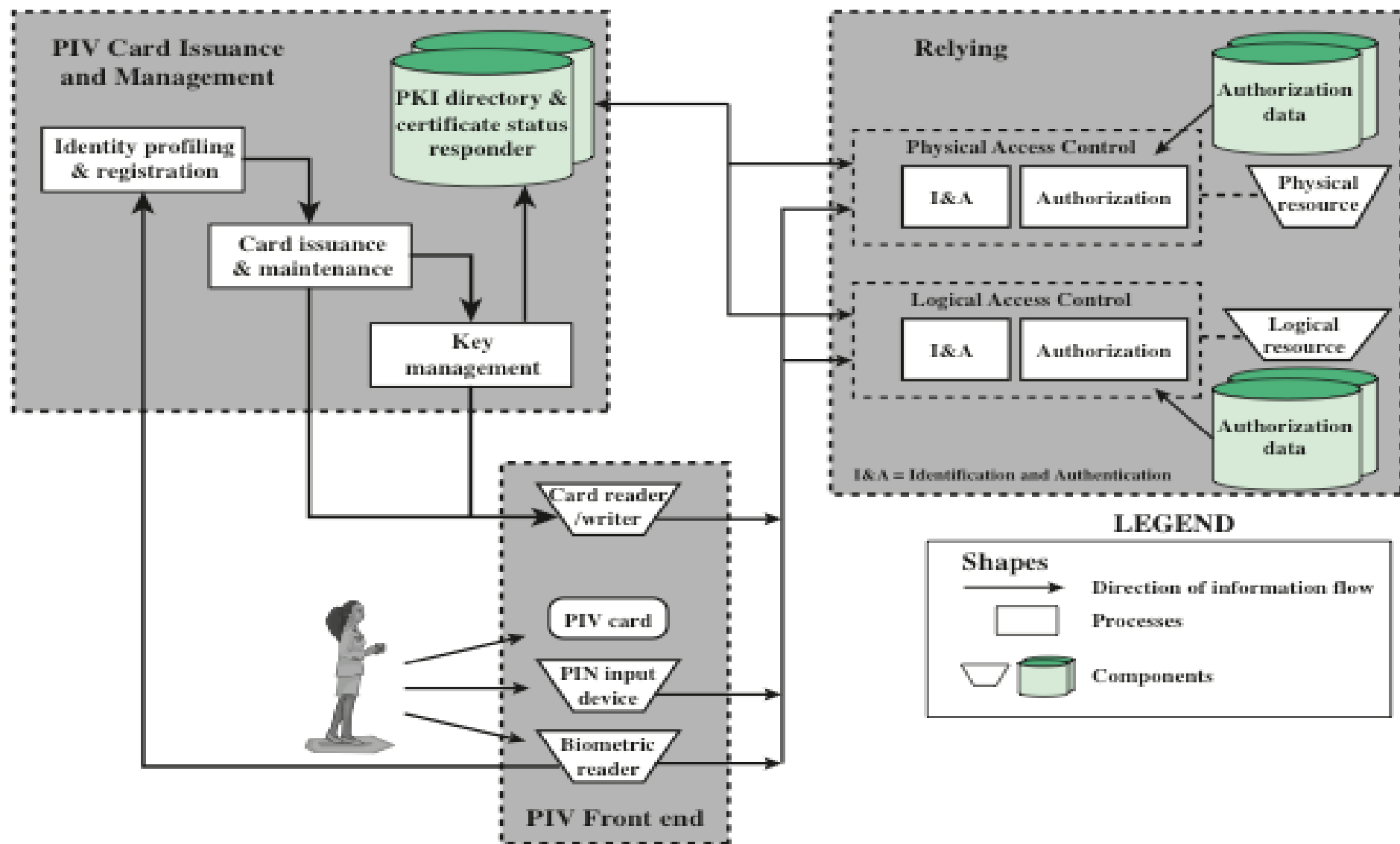


Figure 15.7 FIPS 201 PIV System Model



PIV Credentials and Keys

- › **Personal Identification Number (PIN)**
 - Required to activate the card for privileged operation
- › **Cardholder Unique Identifier (CHUID)**
 - Includes the Federal Agency Smart Credential Number (FASC-N) and the Global Unique Identification Number (GUID), which uniquely identify the card and the cardholder
- › **PIV Authentication Key**
 - Asymmetric key pair and corresponding certificate for user authentication
- › **Two fingerprint templates**
 - For biometric authentication
- › **Electronic facial image**
 - For biometric authentication
- › **Asymmetric Card Authentication Key**
 - Asymmetric key pair and corresponding certificate used for card authentication
- › **Optional elements include the following:**
- › **Digital Signature Key**
 - Asymmetric key pair and corresponding certificate that supports document signing and signing of data elements such as the CHUID
- › **Key Management Key**
 - Asymmetric key pair and corresponding certificate supporting key establishment and transport
- › **Symmetric Card Authentication Key**
 - For supporting physical access applications
- › **PIV Card Application Administration Key**
 - Symmetric key associated with the card management system
- › **One or two iris images**
 - For biometric authentication

PIV Algorithms and Key Sizes

PIV Key Type	Algorithms	Key Sizes (bits)	Application
PIV Authentication Key	RSA	2048	Supports card and cardholder authentication for an interoperable environment.
	ECDSA	256	
Card Authentication Key	3TDEA	168	Supports card authentication for physical access
	AES	128, 192, or 256	
	RSA	2048	Supports card authentication for an interoperable environment.
	ECDSA	256	
Digital Signature Key	RSA	2048 or 3072	Supports document signing and nonce signing
	ECDSA	256 or 384	
Key Management Key	RSA	2048	Supports key establishment and transport.
	ECDH	256 or 384	

Authentication

- › Using the electronic credentials resident on a PIV card, the card supports the following authentication mechanisms:

- **CHUID**

The cardholder is authenticated using the signed CHUID data element on the card. The PIN is not required. This mechanism is useful in environments where a low level of assurance is acceptable and rapid contactless authentication is necessary

- **Card Authentication Key**

The PIV card is authenticated using the Card Authentication Key in a challenge response protocol. The PIN is not required. This mechanism allows contact (via card reader) or contactless (via radio waves) authentication of the PIV card without the holder's active participation, and provides a low level of assurance

- **BIO**

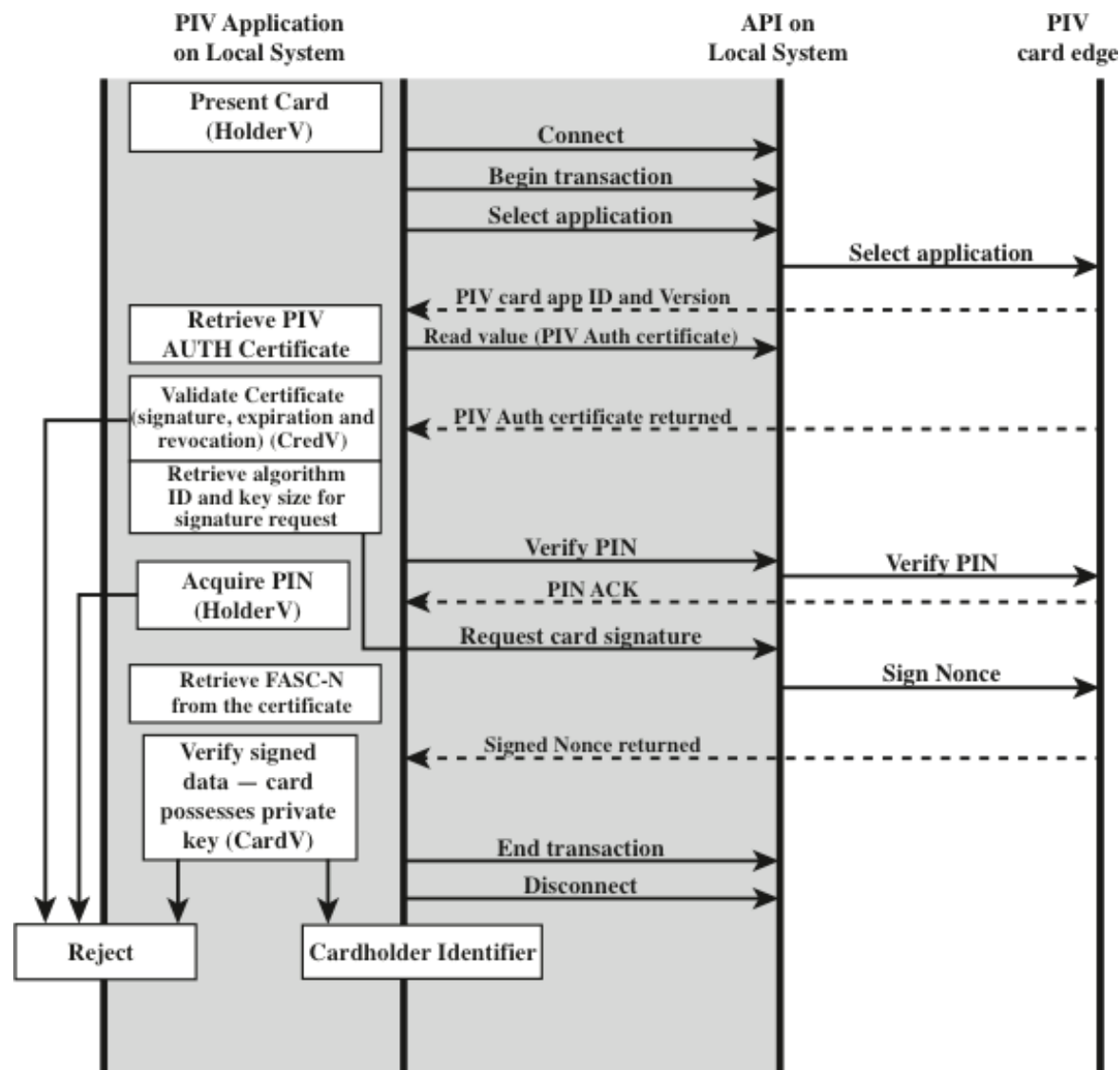
The cardholder is authenticated by matching his or her fingerprint sample(s) to the signed biometric data element in an environment without a human attendant in view. The PIN is required to activate the card. This mechanism achieves a high level of assurance and requires the cardholder's active participation is submitting the PIN as well as the biometric sample

- **BIO-A**

The cardholder is authenticated by matching his or her fingerprint sample(s) to the signed biometric data element in an environment with a human attendant in view. The PIN is required to activate the card. This mechanism achieves a very high level of assurance when coupled with full trust validation of the biometric template retrieved from the card, and requires the cardholder's active participation is submitting the PIN as well as the biometric sample

- **PKI**

The cardholder is authenticated by demonstrating control of the PIV authentication private key in a challenge response protocol that can be validated using the PIV authentication certificate. The PIN is required to activate the card. This mechanism achieves a very high level of identity assurance and requires the cardholder's knowledge of the PIN



CardV = Card validation
 CredV = Credential validation
 HolderV = Cardholder validation
 FASC-N = Federal Agency Smart Credential Number

Figure 15.8 Authentication using PIV Authentication Key